

Les bases de données Graphe

Plan



- Présentation des bases de données Graphe
- Le langage Cypher
- La modélisation



Les bases de données orientées graphes

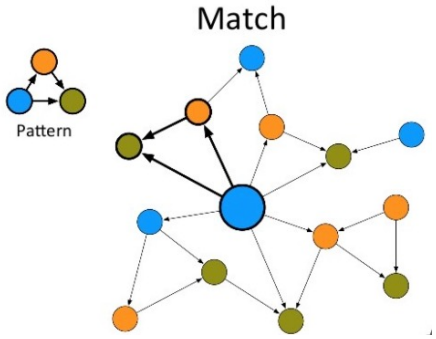
Qu'est-ce qu'un graphe?

Un graphe est un ensemble de points/nœuds reliés entre eux par des arcs.

Les nœuds sont aussi importants que les relations qui les lient.

Les graphes sont partout :

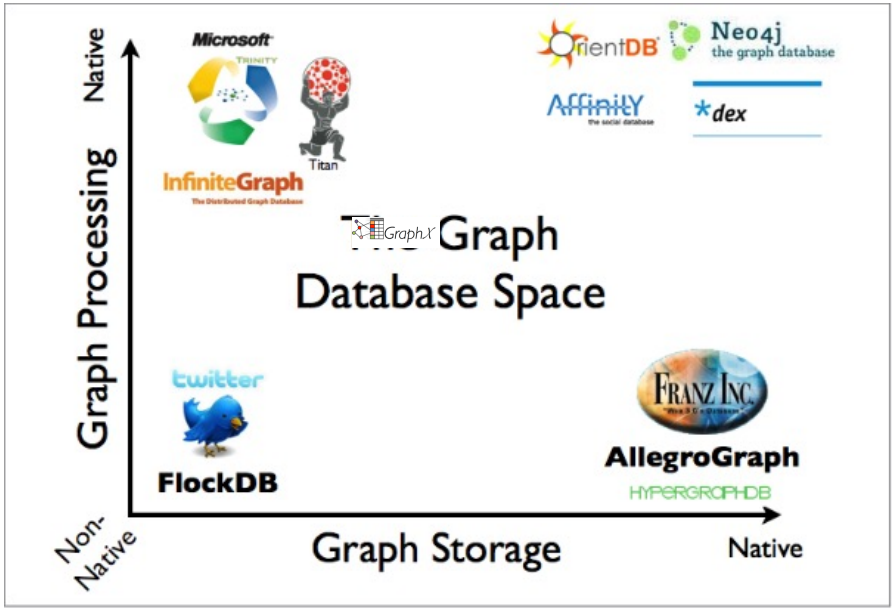
- les réseaux sociaux : Facebook, Meetic …
- le PageRank de Google
- les télécommunications
- Géo-spatial
- IA : la détection de fraude

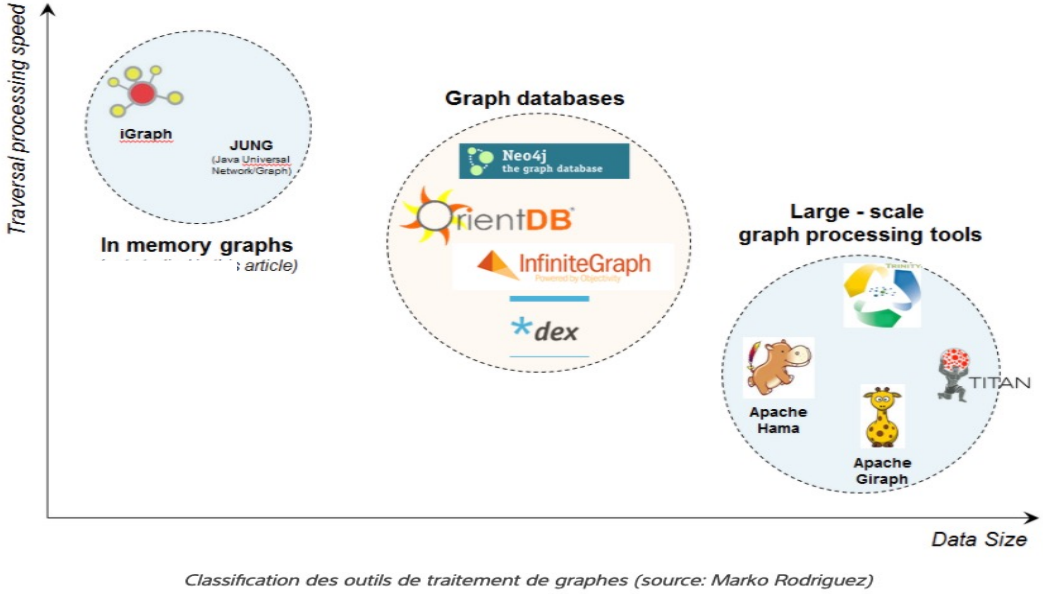


Les bases graphes

Différentes approches de bases de données graphe :

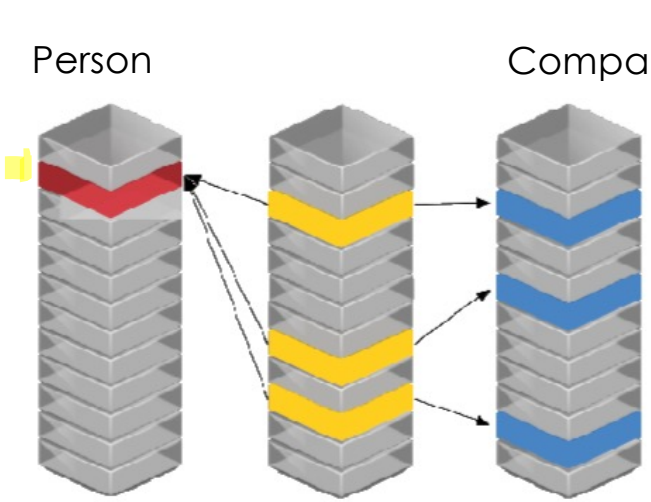
- Un moteur de traitement optimisé pour le parcours des graphes.
- Un stockage natif pour les données graphes.
- Un moteur de traitement et de stockage natif.

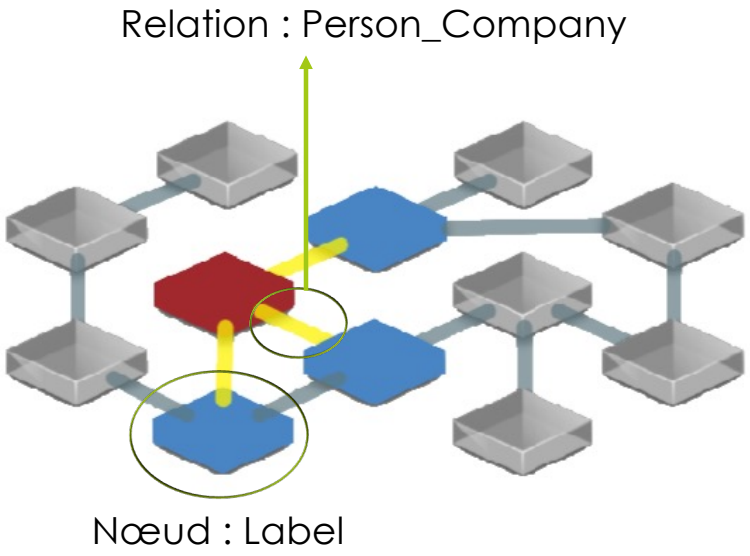




Classification des outils de traitement de graphes (source: Marko Rodriguez)

Les bases graphes VS relationnelles





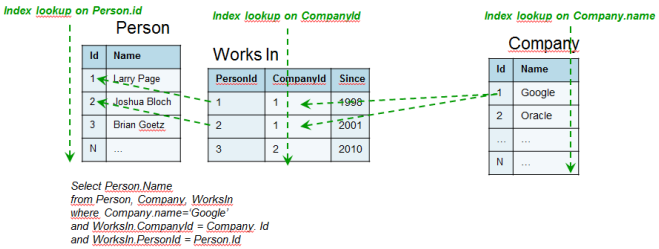
Label : 

Source : http://blog.humancoders.com/interview-florent-biville-formateur-neo4j-pour-human-coders-formations-705/

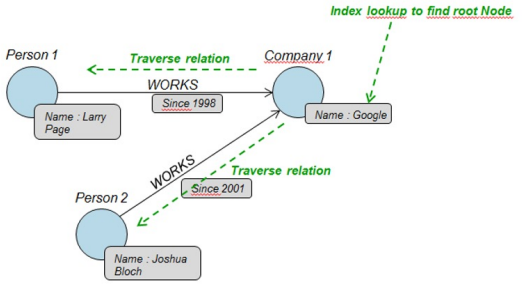
Les bases graphes VS relationnelles

- Recherchons tous les employés de Google ?

SGDBR : Approche ensembliste



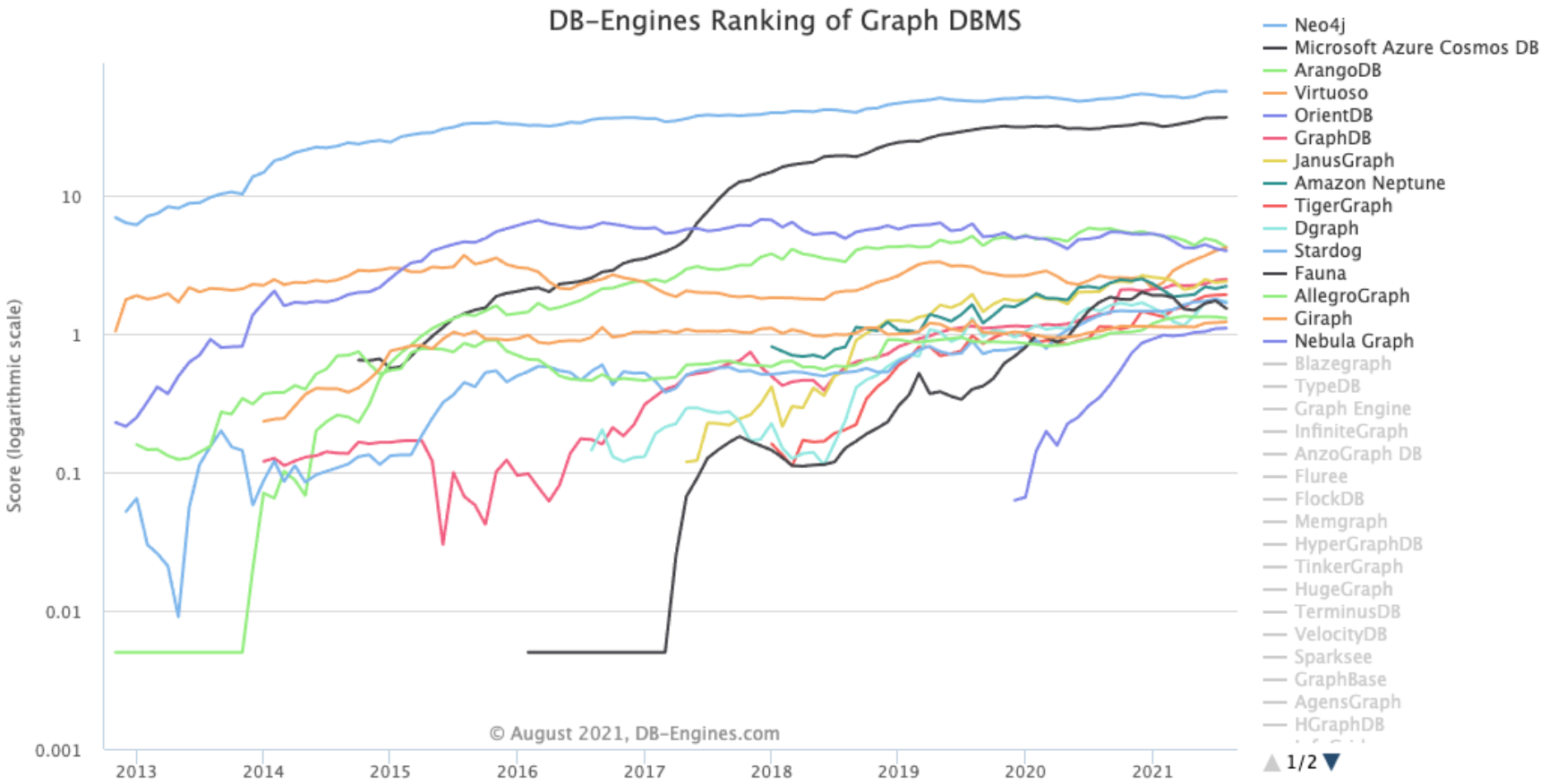
Graphe : Approche par relation



La normalisation accentue le nombre de tables et complexifie les accès aux données à l'inverse de l'approche graphe.

- Que se passe-t'il si on ajoute un nouvel employé à Google ?
- Le modèle graphe, une manière de présenter efficacement et simplement la réalité.

Les bases graphes

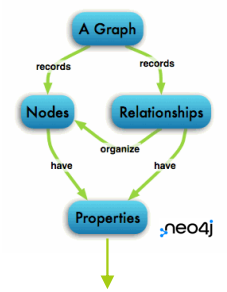


NEO4J

Créé en 2000, aujourd'hui NEO4J est la base de données graphe la plus populaire.

Les API disponibles (Bolt protocol) : Cypher, Python, JAVA, Go, JavaScript, PHP, Perl …

A l'initiative de openCypher : langage orienté graphe basé sur du ASCII Art.



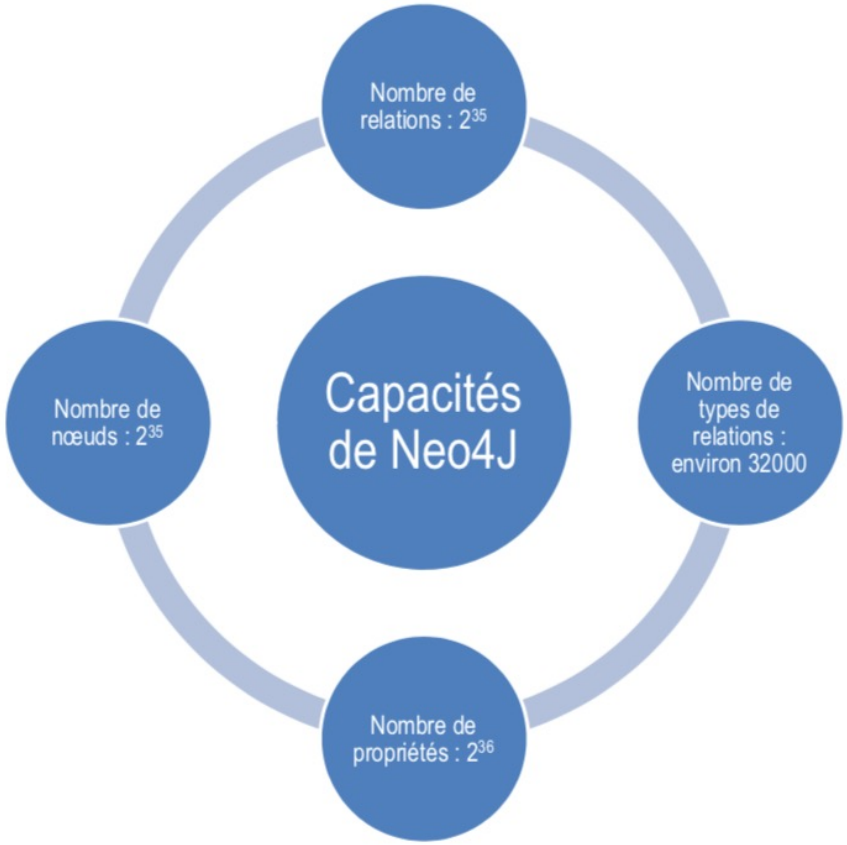
Les types simples :
Integer, Float, String, Boolean, Point, Date, Time, LocalTime, DateTime, LocalDateTime, and Duration.

Les types complexes :
List, map



neo4j

NEO4J


Nombre de relations : 2^{35}
Nombre de nœuds : 2^{35}
Capacités de Neo4J
Nombre de types de relations : environ 32000
Nombre de propriétés : 2^{36}



Cypher

SYNTAXE

Inspirée de:

- **ASCII-Art:**

```
(:Person)-[:LIVES_IN]->(:City)-[:PART_OF]->(:Cou
```

- **SQL:**

Cypher	SQL
MATCH	FROM ... JOIN ON ...
WHERE	WHERE
ORDER BY	ORDER BY
RETURN	SELECT

Syntaxe Cypher

Pour effectuer une requête, les mots clés MATCH / WHERE et RETURN sont la base du langage.

SYNTAXE

Les clauses les plus utilisés sont:

- **MATCH**: sélection de sous-graphes
- **WHERE**: contraintes additionnelles
- **RETURN**: sélection de ce qui est retourné

SYNTAXE

Structure globale d'une requête:

```
MATCH <patterns>
WHERE <conditions>
RETURN <expressions>
```

Exemple:

```
MATCH (director:Person)-[:DIRECTED]->(movie)
WHERE director.name = "Steven Spielberg"
RETURN movie.title
```

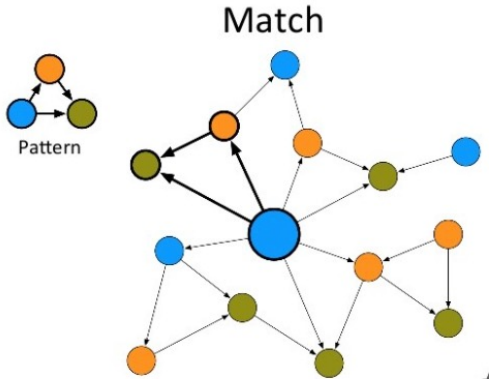
Cypher se veut être un langage simple. La requête ci-dessus recherche les titres des films dont « Steven Spielberg est auteur ».

MATCH

```
MATCH <patterns>
```

Sélectionne le(s) sous-graphe(s) concernés par la requête, en fonction des contraintes exprimées dans les `<patterns>`:

- contraintes structurelles
  - sur les noeuds
  - sur les relations
- contraintes simples (égalité) sur les propriétés (peuvent être exprimées dans la clause `WHERE`)



Syntaxe Cypher : MATCH et les patterns

Le pattern s'exprime sous forme de () pour les nœuds et –[]- > pour les relations . Voyez le comme un moyen d'expliciter des contraintes de recherche sur un ensemble de nœuds ou sur un sous graphe.

Pattern	Description
MATCH (node) RETURN node	Le pattern (node) indique aucune contrainte contrainte de parcours. Le résultat affichera l'ensemble des nœuds et les relations qui les lient.
MATCH (node :Person) RETURN node	Ajoute une contrainte sur les nœuds dont le label est Person. L'ensemble des nœuds satisfaisant la contrainte sera conservé dans la variable node . Le résultat affichera uniquement les nœuds Person et les relations qui peuvent exister entre eux.
MATCH (node :Person { name : 'Steve Spielberg'}) RETURN node	Ajoute une contrainte sur la propriété name des nœuds disposant d'un label Person.
MATCH (node :Person { name : 'Steve Spielberg', birthDate : 1978}) RETURN node	Ajoute une contrainte sur les propriétés name et birthDate.

Syntaxe Cypher : MATCH et les patterns

Pattern	Description
MATCH (p :Person) - - (m :Movie) RETURN p,m	Même si Neo4J nous impose lors de l'insertion des données à indiquer le sens de la relation, il est possible de remonter une relation à contre sens. Cette requête recherche tous les nœuds p avec un label Person directement connecté à un nœud de label Movie sans aucune contrainte sur le type de relation.
MATCH (p :Person) - - > (m :Movie) RETURN p,m	Recherche tous les nœuds p avec un label Person connectés directement quelque soit le TYPE de relation aux nœuds Movie. La relation entre p et On retourne les ensembles p et m.
MATCH (p :Person) – [a:ACTED_IN]- > (m :Movie) RETURN p,a, m	Ajoute une contrainte sur le TYPE de la relation. L'ensemble des relations sortantes de p sera conservé dans une variable a. On retourne les ensembles p,a et m.
MATCH (p :Person) – [a:ACTED_IN {role : 'Neo' }]- > (m :Movie) RETURN p,a, m	Ajoute une contrainte sur la proriété d'une relation.
MATCH (j:Person {name: 'Jennifer'})-[*]-(n) RETURN DISTINCT j, n	Retourne tous les noeuds reliés à "Jennifer" (directement ou indirectement), quelque soit leur distance en nombre de relations.
MATCH (j:Person {name: 'Jennifer'})-[*1..3]-(n) RETURN DISTINCT j, n	On peut définir la profondeur d'un chemin en précisant le nombre de relation à parcourir. Dans cet exemple, on retourne tous les noeuds à une distance de 3 relations maximum autour de "Jennifer".
MATCH (j:Person {name: 'Jennifer'})<-[:PARENT_OF*]-(n) RETURN j, n	Retourne tous les ascendants de "Jennifer".

Syntaxe Cypher : MATCH et les patterns

```
MATCH (a:Actor)-[role:ACTED_IN {roles: ["Neo"] } ]->
                (matrix:Movie {title: "The Matrix"} )
RETURN a.name
```

Retourne le nom de l'acteur qui a joué le rôle de "Neo" dans le film "The Matrix".

```
MATCH (j:Person {name: 'Jennifer'})-[*1..3]-(n)
RETURN DISTINCT j, n
```

Retourne tous les noeuds à une distance de 3 relations maximum autour de "Jennifer".

```
MATCH (j:Person {name: 'Jennifer'})-[*]-(n)
RETURN DISTINCT j, n
```

Retourne tous les noeuds reliés à "Jennifer" (directement ou indirectement), quelque soit leur distance en nombre de relations.

```
MATCH (j:Person {name: 'Jennifer'})<-[:PARENT_OF*]-(n)
RETURN DISTINCT j, n
```

Retourne tous les ascendants de "Jennifer".

WHERE

```
WHERE <conditions>
```

Permet l'expression de contraintes plus complexes que celles de la clause MATCH:

- inégalité
- existance d'une propriété
- test sur valeur partielle
- test sur sous-graphe

(c'est l'équivalent de la clause WHERE du SQL)

WHERE : ÉGALITÉ

```
MATCH (tom:Person)
WHERE tom.name = "Tom Hanks"
RETURN tom
```

```
MATCH (tom:Person {name:'Tom Hanks'})
RETURN tom
```

WHERE : INÉGALITÉ

```
MATCH (p:Person)
WHERE NOT p.name = 'Tom Hanks'
RETURN p
```

```
MATCH (nineties:Movie)
WHERE nineties.released > 1990 AND nineties.released < 2000
RETURN nineties.title
```

WHERE : EXISTANCE D'UNE PROPRIÉTÉ

```
MATCH (p:Person)
WHERE exists(p.birthdate)
RETURN p.name
```

Retourne les personnes ayant une date d'anniversaire.

```
MATCH (p:Person)-[rel:WORKS_FOR]->(:Company)
WHERE exists(rel.startYear)
RETURN p
```

Retourne les personnes ayant une date de début de contrat.

WHERE : VALEUR PARTIELLE

```
MATCH (tom:Person)
WHERE tom.name STARTS WITH 'Tom'
RETURN tom
```

Retourne les acteurs dont le prénom commence par "Tom".

```
MATCH (p:Person)
WHERE p.name CONTAINS 'k'
RETURN p
```

Retourne les acteurs dont le nom contient la lettre "k".

WHERE : VALEUR PARTIELLE

```
MATCH (p:Person)
WHERE p.name =~ 'Tom.*'
RETURN p.name
```

```
MATCH (p:Person)
WHERE p.yearsExperience IN [1, 5, 6]
RETURN p.name, p.yearsExperience
```

WHERE : SOUS-GRAPHE

```
MATCH (p:Person {name:'Tom'})-[r:IS_FRIENDS_WITH]->(friend:Person)
WHERE exists((friend)-[:WORKS_FOR]->(:Company {name:'Neo4j'}))
RETURN friend
```

Retourne les amis de Tom qui travaillent pour la société Neo4j.

```
MATCH (p:Person {name:'Tom'})-[r:IS_FRIENDS_WITH]->(friend:Person)
-[:WORKS_FOR]->(:Company {name:'Neo4j'})
RETURN friend
```

Les 2 requêtes donnent exactement le même résultat.

RETURN : MOTS-CLÉS ASSOCIÉS

```
RETURN [DISTINCT] <expressions>
[ ORDER BY <aliases> [DESC] ]
[ LIMIT <nombre de lignes> ]
```

Le résultat d'une requête peut être contrôlé par quelques mots-clés associés à la clause `RETURN`:

- DISTINCT
- ORDER BY
- LIMIT

RETURN : DISTINCT

```
MATCH (p)-[:LIKES]-(t:Technology)
WHERE t.type IN ['Graphs','Query Languages']
RETURN p.name
```

RETURN : LIMIT

Limite la taille de la réponse au nombre de lignes spécifié.

Si le résultat n'est pas trié, le choix des lignes retournées n'a pas de signification particulière.

```
MATCH (p)-[:LIKES]-(t:Technology)
RETURN p.name, count(t) AS numberOfTechnologies
ORDER BY numberOfTechnologies
LIMIT 3
```

RETURN : ORDER BY

Trie le résultat final (`RETURN`) ou intermédiaire (`WITH`).

(même syntaxe que la clause `ORDER BY` du SQL)

```
MATCH (p)-[:LIKES]-(t:Technology)
RETURN p.name, count(t) AS numberOfTechnologies
ORDER BY p.name DESC, numberOfTechnologies
```

Let’s graph



Les CRUD !!!!



CRUD

Dans cette section nous aborderons la syntaxe CYPHER pour :

- insérer des nœuds et des relations.
- mettre à jour des propriétés sur des nœuds et des relations.
- supprimer des nœuds, des relations et des propriétés.

Notez que la recherche de données a été abordé dans la section précédente :

CRUD

Opération	SQL	CYPHER
Create	INSERT	CREATE / MERGE [SET]
Read	SELECT	RETURN
Update	UPDATE	SET
Delete	DELETE	[DETACH] DELETE

CREATE

```
[ MATCH <patterns> ]
[ WHERE <conditions> ]
CREATE <patterns>
[    SET <expressions> ]
[ RETURN <expressions> ]
```

MATCH/WHERE s'utilisent comme dans une requête en lecture:

- pour relier 2 noeuds existants par une nouvelle relation
- pour obtenir les informations nécessaires à la création des nouveaux objets

CRUD : création d'un nœud, d'une relation

Création d'un nœud avec une proriété :

```
CREATE (m:Person {name: 'Michael'})
RETURN m
```

La clause RETURN est optionnelle.

Création d'une relation à partir de 2 nœuds existants :

```
MATCH (j:Person {name: 'Jennifer'})
MATCH (m:Person {name: 'Michael'})
CREATE (j)-[rel:IS_FRIENDS_WITH {since:2018}]->(m)
```

Une relation est obligatoirement liée à deux nœuds.

Création de 2 nœuds (Jennifer et Michael) avec une relation de type `IS_FRIENDS_WITH` de Jennifer vers Michael.

```
CREATE (j:Person {name: 'Jennifer'}) -[rel:IS_FRIENDS_WITH]->(m:Person
{name: 'Michael'})
```

Une autre manière de formuler la création précédente :

```
CREATE (j:Person {name: 'Jennifer'})
       (m:Person {name: 'Michael'})
       (j)-[rel:IS_FRIENDS_WITH]->(m)
```

CRUD : création d'une propriété

2 syntaxes possibles:

- soit dans le <pattern> (plus concis)
- soit avec la clause SET (plus riche, prioritaire

```
CREATE (j:Person{name: 'Jennifer', })-[rel:IS_FRIENDS_WITH{since :
2018}]->(m:Person{name:'Michael'})
```

```
CREATE (j:Person)-[rel:IS_FRIENDS_WITH]->(m:Person{name:'Michael'})
SET j.name = 'Jennifer', rel.since = 2018
```

CRUD : MERGE

La syntaxe de MERGE est identique à celle de CREATE :

[**MATCH** <patterns>]
[**WHERE** <conditions>]
MERGE <patterns>
    [**SET** <expressions>]
[**RETURN** <expressions>]

```
MERGE (m:Person {name: 'Michael'})
RETURN m
```

Son comportement est différent de celui de CREATE:

- si il existe, alors le noeud existant est retourné
- sinon le noeud créé est retourné

CRUD : MERGE

Son comportement est différent de celui de CREATE:
- si la relation existe, alors elle est retournée
- sinon la relation créée est retournée

Si Jennifer et/ ou Michael n'existe pas ?

Neo4J ne crée pas la relation

```
MATCH (j:Person {name: 'Jennifer'})
MATCH (m:Person {name: 'Michael'})
MERGE (j)-[r:IS_FRIENDS_WITH]->(m)
RETURN j, r, m
```

```
MERGE (j:Person {name: 'Jennifer'}) -[r:IS_FRIENDS_WITH]-
>(m:Person {name: 'Michael'}) RETURN j, r, m
```

```
MERGE (a:Person {name: "Jennifer"})
MERGE (b:Person {name: "Michael"})
MERGE (a)-[:IS_FRIENDS_WITH]->(b)
```

ATTENTION: Cette requête peut créer des duplicats !

C'est l'existence du sous-graphe entier (et non de chacun de ses noeuds/relations) qui est vérifiée.

Par conséquent, l'absence de la relation r a pour conséquence la création de la relation et des 2 noeuds, même si ceux-ci existent déjà.

CRUD : MERGE

```
MERGE (m:Person {name: 'Michael'}) -[r:IS_FRIENDS_WITH]-
(j:Person {name:'Jennifer'}) SET r.since = date('2018-03-01')
```

La clause SET est exécutée dans tous les cas:

- si la relation existe, alors la propriété r.since de cette relation est modifiée
- sinon la relation est créée, et sa propriété r.since est initialisée

```
MERGE (m:Person {name: 'Michael'}) -[r:IS_FRIENDS_WITH]-
(j:Person {name:'Jennifer'})
    ON CREATE SET r.since = date('2018-03-01')
    ON MATCH SET r.updated = date()
RETURN m, r, j
```

Permet de contrôler le comportement souhaité pour la clause SET:

- ON CREATE SET n'est exécuté qu'en cas de création d'une nouvelle relation
- ON MATCH SET n'est exécuté que si la relation existait déjà

CRUD : SET

CREATE | **MERGE** | **MATCH** <patterns>
SET <variable>.<propriété> = <valeur>

Affecte une nouvelle valeur aux propriétés des objets sélectionnés ou créés.

- ajout de propriétés
- mise à jour de propriétés
- suppression de propriétés

```
MATCH (j:Person {name: 'Jennifer'}) -[rel:WORKS_FOR]-(:Company
{name: 'Neo4j'})
SET j.birthdate = date('1980-01-01'), rel.startYear =
date({year: 2018})
RETURN p, rel
```

Si le nœud existe, alors la clause SET est exécutée dans les 2 cas:

- si la propriété existe, elle est modifiée
- sinon elle est créée et initialisée

CRUD : DELETE

MATCH <patterns>
[DETACH] DELETE <variable>

Permet de supprimer différents types d'objet:

- noeud
- relation
- sous-graphe

```
MATCH (m:Person {name: 'Michael'})
DELETE m
```

Provoque une erreur si ce nœud a toujours des relations, car Neo4j est ACID-compliant.

```
MATCH (j:Person {name: 'Jennifer'}) -[r:IS_FRIENDS_WITH]->(m:Person {name: 'Michael'})
DELETE r
```

Supprime la relation r

CRUD : DELETE

```
MATCH (m:Person {name: 'Michael'})
DETACH DELETE m
```

Supprime les relations du noeud m et le noeud lui-même

```
MATCH (n:Person {name:
'Jennifer'}) REMOVE n.birthdate
```

```
MATCH (n:Person {name:
'Jennifer'}) SET n.birthdate =
null
```

Neo4j ne stocke pas de valeur nulle.

Une propriété de valeur nulle équivaut donc à l'absence de cette propriété (schema-less).

Let’s graph

